Assignment No. 3

Sanskruti Dahiphale
123B1B007

1. Problem Statement:

You find yourself in a labyrinthine dungeon with multiple rooms and connecting passageways. Each passage has a specific time to travel, and some passages are blocked due to cave-ins. Starting from the main entrance, you must find the shortest escape routes to every other room in the dungeon. Use Dijkstra's algorithm to determine the quickest time required to reach each room from the entrance. If any room is isolated and unreachable, mark it as -1. Test this with a dungeon of at least 15 rooms and random travel times for each accessible passage.

2. Prerequisite:

Knowledge of greedy strategy

3. Objectives:

- To implement single-source shortest path algorithms.
- To model real-world navigation and routing problems using graphs.

4. Course Outcome Mapping:

CO2 - Analyze the efficiency of algorithm using either Divide and Conquer or Greedy strategy.

5. Theory

1. Dijkstra’s Algorithm

It is a shortest path algorithm used in graph theory to find the minimum distance from a starting node to all other nodes in a weighted graph. The algorithm works only when all edge weights are non-negative.

In this assignment, the dungeon is represented as a graph where:

- Rooms are considered as vertices (nodes).
- Passageways between rooms are considered as edges.
- Travel time between rooms represents the weight of the edge.

The goal is to start from the main entrance (source node) and find the minimum time required to reach every other room. If a room has no path from the entrance, it is considered unreachable and marked as -1.

Dijkstra's algorithm repeatedly selects the nearest unvisited node and updates the distances to its neighboring nodes until all nodes are processed.

This algorithm is widely used in applications such as:

- Network routing
- GPS navigation systems
- Pathfinding in games
- Transportation planning

2. Greedy Strategy

Dijkstra's algorithm follows a Greedy Strategy.

A greedy algorithm makes the best possible choice at the current step without worrying about future consequences.

In Dijkstra's algorithm:

- At each step, the algorithm chooses the node with the smallest temporary distance from the source.
- Once that node is selected, its distance becomes final because no shorter path will be found later.

This greedy decision ensures that the algorithm gradually builds the shortest path tree from the source node.

Edge Relaxation

A key concept used in Dijkstra's algorithm is edge relaxation.

Edge relaxation means updating the shortest known distance to a node if a shorter path is found through another node.

For example:

If the current distance to room B is 10 minutes, and going through room A gives:

Distance(Source → A) + Distance(A → B) = 6 minutes

Then the distance to B is updated from 10 to 6.

This process is repeated until all nodes have the minimum possible distance.

Priority Queue

For optimization, Dijkstra's algorithm often uses a priority queue.

A priority queue always gives the node with the smallest distance first. This reduces unnecessary comparisons and makes the algorithm faster for large graphs.

However, in smaller implementations or simple programs, the algorithm can also be implemented using arrays or lists.

Example

Consider a small dungeon with **5 rooms**.

Room Connections with Time:

0 → 1 (4)
0 → 2 (2)
1 → 3 (5)
2 → 1 (1)
2 → 3 (8)
3 → 4 (3)

Entrance = **Room 0**

Step 1

Distance from source:

Room 0 = 0
Room 1 = ∞
Room 2 = ∞
Room 3 = ∞
Room 4 = ∞

Step 2

From Room 0:

Room 1 = 4
Room 2 = 2

Step 3

Choose the smallest distance node → **Room 2**

Check neighbors:

Room 2 → Room 1

New distance = 2 + 1 = 3
Update Room 1 (4 → 3)
Room 2 → Room 3
New distance = 2 + 8 = 10
Update Room 3 = 10

Step 4

Choose next smallest → **Room 1**

Room 1 → Room 3
New distance = 3 + 5 = 8
Update Room 3 (10 → 8)

Step 5

Choose next smallest → **Room 3**

Room 3 → Room 4
New distance = 8 + 3 = 11

Final Distances

Room 0 = 0
Room 1 = 3
Room 2 = 2
Room 3 = 8
Room 4 = 11

This means the **shortest time from the entrance to each room** has been calculated.

Algorithm: Karatsuba Multiplication

Input: Step 1: Represent the dungeon as a graph with rooms as nodes and passageways as edges.

Step 2: Initialize a distance array where:

- Distance of source node = 0
- Distance of all other nodes = ∞

Step 3: Mark all nodes as unvisited.

Step 4: Select the unvisited node with the **smallest distance**.

Step 5: For each neighboring node:

- Calculate the new distance.
- If the new distance is smaller, update it.

Step 6: Mark the selected node as visited.

Step 7: Repeat steps 4–6 until all nodes are visited.

Step 8: If any node still has distance ∞, mark it as **-1 (unreachable)**.

Time Complexity Analysis

The time complexity of Dijkstra's algorithm depends on the data structure used.

Using Simple Array / List

To find the minimum distance node we scan all nodes.

Time Complexity: $O(V^2)$

Where: V = number of vertices (rooms)
This approach is simpler and suitable for small graphs.

Using Priority Queue (Min Heap)

When a priority queue (heap) is used: O((V + E) log V)

Where: V = vertices
E = edges
This is much faster for large graphs because it avoids scanning all vertices repeatedly.

6. Implementation

```
import heapq

import random

# Number of rooms

n = 15

# Create an empty graph

graph = {i: [] for i in range(n)}

# Randomly create passages between rooms

for i in range(n):

    for j in range(i + 1, n):

        # Randomly decide if passage exists

        if random.choice([True, False]):
```

```
            time = random.randint(1, 20)  # travel time
            graph[i].append((j, time))
            graph[j].append((i, time))  # undirected graph



# Dijkstra Algorithm
def dijkstra(graph, start, n):
    # Distance array
    dist = [float('inf')] * n
    dist[start] = 0
   # Priority queue
    pq = [(0, start)]
     while pq:
        current_dist, node = heapq.heappop(pq)

        for neighbor, weight in graph[node]:
            new_dist = current_dist + weight

            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))
# Mark unreachable rooms as -1
    for i in range(n):
        if dist[i] == float('inf'):
            dist[i] = -1
return dist
# Start from entrance room 0
result = dijkstra(graph, 0, n)
```

```
# Print dungeon passages
print("Dungeon Passages (Room -> Neighbor, Time):")
for room in graph:
    print(room, "->", graph[room])
print("\nShortest Time from Entrance (Room 0):")
for i in range(n):
    print(f"Room {i} : {result[i]}")
```

7. Sample Input & Output

```
pccoe@pc-9:~$ cd DAA
pccoe@pc-9:~/DAA$ nano assign3.py
pccoe@pc-9:~/DAA$ python3 assign3.py
Dungeon Passages (Room -> Neighbor, Time):
0 -> [(2, 3), (4, 10), (6, 9), (10, 2), (11, 18)]
1 -> [(2, 6), (8, 16), (10, 17), (11, 2), (12, 6), (14, 15)]
2 -> [(0, 3), (1, 6), (3, 16), (4, 16), (6, 14), (8, 3), (9, 20), (11, 14), (14, 6)]
3 -> [(2, 16), (4, 2), (5, 8), (6, 13), (7, 14), (9, 4), (10, 6), (11, 4), (13, 9)]
4 -> [(0, 10), (2, 16), (3, 2), (6, 10), (7, 19), (9, 5), (10, 17)]
5 -> [(3, 8), (8, 18), (10, 9), (11, 14), (12, 17), (13, 15), (14, 3)]
6 -> [(0, 9), (2, 14), (3, 13), (4, 10), (10, 14), (13, 2), (14, 10)]
7 -> [(3, 14), (4, 19), (8, 16), (9, 3), (10, 17), (11, 5), (12, 19), (13, 4), (14, 6)]
8 -> [(1, 16), (2, 3), (5, 18), (7, 16), (9, 7), (13, 18)]
9 -> [(2, 20), (3, 4), (4, 5), (7, 3), (8, 7), (12, 12), (13, 8), (14, 12)]
10 -> [(0, 2), (1, 17), (3, 6), (4, 17), (5, 9), (6, 14), (7, 17), (11, 18), (12, 3)]
11 -> [(0, 18), (1, 2), (2, 14), (3, 4), (5, 14), (7, 5), (10, 18), (12, 8), (14, 2)]
12 -> [(1, 6), (5, 17), (7, 19), (9, 12), (10, 3), (11, 8), (14, 20)]
13 -> [(3, 9), (5, 15), (6, 2), (7, 4), (8, 18), (9, 8)]
14 -> [(1, 15), (2, 6), (5, 3), (6, 10), (7, 6), (9, 12), (11, 2), (12, 20)]

Shortest Time from Entrance (Room 0):
Room 0 : 0
Room 1 : 9
Room 2 : 3
Room 3 : 8
Room 4 : 10
Room 5 : 11
Room 6 : 9
Room 7 : 15
Room 8 : 6
Room 9 : 12
Room 10 : 2
Room 11 : 11
Room 12 : 5
Room 13 : 11
Room 14 : 9
pccoe@pc-9:~/DAA$
```

8. Conclusion

In this assignment, Dijkstra's algorithm was used to determine the **shortest escape route from the entrance to all other rooms in a dungeon**. The dungeon was modeled as a **weighted graph**, where rooms were vertices and passageways were edges with travel times.

The algorithm works using a **greedy strategy**, repeatedly selecting the nearest unvisited room and updating distances using **edge relaxation**. If any room cannot be reached from the entrance due to blocked passages, it is marked as **-1**.

The implementation successfully calculates the shortest travel time for each room in a dungeon containing **at least 15 rooms with random connections**.

The time complexity of the implementation using arrays is **$O(V^2)$**, which is efficient enough for smaller graphs.

Thus, Dijkstra's algorithm provides an effective method for solving shortest path problems in weighted graphs such as dungeon navigation or network routing.